

# Decisões de Arquitetura

Este documento detalha todas as decisões arquiteturais, os protocolos de comunicação e os algoritmos distribuídos utilizados no projeto. O objetivo principal é servir como roteiro teórico para a apresentação, vinculando cada funcionalidade implementada diretamente aos conceitos ministrados nos slides da disciplina de Sistemas Distribuídos.

## 1. Arquitetura de Sistema e de Software

A organização dos processos e componentes é a base estrutural do sistema. Optou-se por uma arquitetura flexível que transita entre descentralização e centralização para resolver os conflitos de estado no jogo.

| Conceito                              | Aplicação no Projeto                                                                                                                                                            | Justificativa Teórica (Fonte)                                                                                                                                                     |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Arquitetura de Sistema Híbrida</b> | O sistema opera de forma mista. Inicia descentralizado (Peer-to-Peer - P2P) onde todos os nós são iguais para eleger o líder. Após a eleição, muda para Cliente-Servidor (C/S). | Combina a tolerância a falhas do P2P com a facilidade de gerenciar o estado global (os votos e o tempo) inerente ao C/S. <i>Ref: Módulo 2 - Aula 4 (Arquiteturas de Sistema).</i> |
| <b>Servidor com Estado (Stateful)</b> | O nó "Cérebro" armazena o estado atual dos cofres, papéis dos jogadores e o histórico do chat.                                                                                  | Necessário para validar os turnos e recuperar o jogo em caso de queda de clientes (reatribuição do Infiltrado). <i>Ref: Módulo 3 - Aula 5 (Servidores com e sem estado).</i>      |

## 2. Protocolos de Comunicação

A comunicação entre processos remotos dita como as ações viajam pela rede. Foram tomadas decisões explícitas tanto na camada de transporte quanto na camada de aplicação.

- **Camada de Transporte:** Utilização estrita de **Sockets TCP**. Justifica-se pela necessidade de um serviço orientado a conexão e de transferência de dados confiável (em ordem e sem perdas). Se um voto fosse perdido, a mecânica do jogo seria arruinada. (*Ref: Módulo 4 - Parte 1 - Aula 1 e Aula 3*)
- **Comunicação Transiente e Assíncrona:** O sistema de chat envia mensagens

transientes. O nó não guarda mensagens caso o destinatário esteja offline antes da conexão. Além disso, as chamadas de I/O são assíncronas (via Threads) para permitir que o usuário digite enquanto recebe cotações simultaneamente. (Ref: Módulo 4 - Parte 2 - Aula 1)

## 2.1. Protocolo de Aplicação Inventado

Em atendimento ao requisito do trabalho, desenvolvemos um protocolo proprietário formatado em texto estruturado. Os delimitadores facilitam o parsing (Marshalling/Unmarshalling) das ações. (definido ao final desse documento)

Formato Geral:

[TIPO\_MENSAGEM][ARGUMENTOS\_SEPARADOS\_POR\_DOIS\_PONTOS]

## 3. Concorrência e Multitarefa

Tanto o Cliente quanto o Servidor devem lidar com o envio de dados do teclado e o recebimento de mensagens da rede simultaneamente.

| Componente                  | Implementação e Justificativa                                                                                                                                                                                                                         |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Servidor Concorrente</b> | O "Cérebro" possui uma <i>Dispatcher Thread</i> (que aceita conexões <code>`accept()`</code> ) e aloca uma <i>Worker Thread</i> para cada cliente. Isso impede chamadas de sistema bloqueantes de congelarem todo o processo. Ref: Módulo 3 - Aula 5. |
| <b>Cliente Multitarefa</b>  | O processo Cliente possui duas Threads: uma aguardando o <code>`input()`</code> do terminal (bloqueante) e outra em loop aguardando o <code>`recv()`</code> do socket para imprimir o chat. Permite sobreposição de processamento e comunicação.      |

## 4. Algoritmos Distribuídos: Coordenação e Sincronização

### 1. Eleição de Líder - Algoritmo de Intimidação (Bully):

- **Cenário:** Sem servidor fixo, o processo com maior ID detecta a ausência de um coordenador e envia mensagens ``ELECTION``. Se ninguém com ID maior responde, ele assume via ``LEADER``.
- **Justificativa:** É fundamental para o dinamismo e a configuração ad-hoc da

arquitetura híbrida. (Ref: Módulo 6 - Parte 2 - Aula 2)

## 2. Ordenação de Eventos - Multicast com Entrega Causal:

- **Cenário:** No chat, a comunicação ocorre via Multicast provido pela camada de aplicação do Cérebro.
- **Implementação:** Usamos **Vetores de Timestamp (VT)** em cada mensagem. O sistema avalia o VT da mensagem contra o VT local. Se a mensagem do jogador B for uma resposta ao jogador A, o sistema retarda a entrega da mensagem de B até que a de A tenha chegado, garantindo a lógica da conversa.
- **Justificativa:** Ordem causal é imprescindível em interações humanas distribuídas (como um jogo de dedução), garantindo que as relações de causa-efeito não quebrem por atrasos da rede. (Ref: Módulo 6 - Parte 2 - Aula 1)

# Fluxo do Jogo

## O Jogo: "A Palavra Infiltrada"

**O Conceito:** O jogo é inspirado no board game *Undercover*. A maioria dos jogadores recebe uma palavra secreta em comum (Ex: "Maçã"). Um jogador (o Infiltrado) recebe uma palavra muito parecida, mas diferente (Ex: "Pêra"). Ninguém sabe quem é quem. Eles precisam descobrir quem está com a palavra errada.

## O Fluxo da Rodada (Passo a Passo)

### 1. Descoberta e Eleição (O Início)

- Todos abrem o terminal e rodam `python jogo.py`.
- Os processos se descobrem na rede e usam o **Algoritmo de Intimidação (Bully)** para eleger o "Host" (Líder).
- O Host abre uma porta TCP (usando **Threads** para ouvir múltiplos clientes) e os outros se conectam a ele.

### 2. A Distribuição (Unicast)

- O Host tem uma lista fixa de pares de palavras no código (ex: [ "Guitarra", "Baixo" ]).
- Ele sorteia secretamente quem será o Infiltrado.
- O Host usa o socket TCP para mandar uma mensagem privada para cada um.
  - Para os normais: `ROLE|PALAVRA:Guitarra`
  - Para o Infiltrado: `ROLE|PALAVRA:Baixo`

### 3. A Dica Inicial

- O terminal de todos pede: *"Digite uma palavra relacionada para provar que você sabe a palavra secreta:"*.
- Cada jogador envia sua dica para o Host.
- O Host espera receber a dica de todos, agrupa tudo e faz um **Multicast** para todos verem a lista.
  - *Tela:* \* João disse: "Cordas"
    - Maria disse: "Pesado" (Maria é a infiltrada com 'Baixo')
    - Pedro disse: "Palheta"

### 4. A Fase de Chat e Sincronização Causal

- O Host envia a mensagem: *"Vocês têm 60 segundos para discutir. Para pular para a votação, digite /votar"*.

- Aqui entra o **Chat com Vetores de Timestamp (Módulo 6)**. Toda mensagem enviada no chat carrega o vetor para garantir que uma resposta não chegue antes da pergunta na tela dos colegas.
- **A Barreira de Sincronização (A mecânica nova)**: O Host mantém uma variável de estado `votos_skip = 0`.
  - Se o João digitar `/votar`, o Host anota `votos_skip = 1` e manda um Multicast avisando: *"João quer votar (1/3)"*.
  - Se a Maria e o Pedro também digitarem `/votar`, o `votos_skip` chega a 3 (o total de jogadores).
  - O Host imediatamente aborta o relógio de 60 segundos e envia o comando `END_CHAT` por Multicast, travando o chat de todos.

## 5. A Votação Final

- A tela pede: *"Quem é o Infiltrado? Digite o nome."*
- Todos mandam seu voto em segredo para o Host.
- O Host calcula os votos e manda o veredito final: *"Maria era a Infiltrada e foi eliminada! Os inocentes venceram!"*.

## Fluxo de Falhas

1. **Caiu um Jogador Comum:**
  - O Cérebro detecta a queda no `recv()`.
  - Reduz a variável de `jogadores_ativos`.
  - Recalcula a barreira do `/votar` (se precisava de 4, agora precisa de 3).
  - Manda Multicast: *"O jogador X perdeu a conexão. A discussão continua."*
2. **Caiu o Infiltrado:**
  - O Cérebro detecta a queda e vê no estado que ele era o Infiltrado.
  - Manda Multicast: *"Atenção! O Infiltrado fugiu da rede! Rodada anulada. Sorteando nova palavra..."*
  - O Cérebro zera o chat, não dá ponto pra ninguém, sorteia uma palavra nova e recomeça a Fase 1 da rodada.
3. **Caiu o Cérebro:**
  - Os clientes detectam a falha.
  - Inicia a **Eleição Bully**.
  - O vencedor grita: *"Eu sou o novo Cérebro!"*.
  - Como o Cérebro não sabe a palavra secreta antiga (estava só na memória RAM do Cérebro morto), ele simplesmente avisa: *"O Cérebro foi interceptado. Anulando rodada. Sorteando nova palavra..."* e a partida recomeça mantendo o placar.

# Protocolo de Aplicação

Este documento define o formato das mensagens que transitarão na camada de aplicação via sockets TCP e UDP, conforme o Módulo 4 - Parte 1. O protocolo adota uma estrutura em texto plano (string) utilizando delimitadores lógicos.

## Padrão Geral de Empacotamento

O formato geral segue a estrutura: COMANDO|CHAVE1:VALOR1|CHAVE2:VALOR2

- O caractere | separa o comando dos seus atributos.
- O caractere : funciona como chave-valor para cada argumento.

---

## Fase 1: Eleição Bully (Descentralizada - UDP/TCP)

| Mensagem                                        | Direção                    | Significado                                                 |
|-------------------------------------------------|----------------------------|-------------------------------------------------------------|
| ELECTION ID:<meu_id>                            | Nó -> Nós de ID maior      | Inicia a eleição avisando que o Cérebro caiu ou não existe. |
| ELECTION_OK                                     | Nó Maior -> Nó Menor       | Aviso para o nó menor abortar a sua eleição.                |
| COORDINATOR ID:<novo_lider_id> PORT:<porta_tcp> | Líder -> Todos (Broadcast) | Anuncia a vitória e informa a porta para conexão C/S.       |

---

## Fase 2: Gestão do Jogo (Cérebro <-> Clientes via TCP)

| Mensagem                                       | Direção                       | Significado                                         |
|------------------------------------------------|-------------------------------|-----------------------------------------------------|
| JOIN NAME:<nome_jogador>                       | Cliente -> Cérebro            | Entrar no lobby do jogo após a eleição.             |
| GAME_START                                     | Cérebro -> Todos              | Sinaliza que o jogo vai começar.                    |
| ROLE ROLE:<INOCENTE/INFILTRADO> WORD:<palavra> | Cérebro -> Cliente Específico | Informa secretamente o papel e a palavra da rodada. |
| TIP_REQ                                        | Cérebro -> Todos              | Pede para todos digitarem sua dica inicial.         |
| TIP WORD:<palavra_digitada>                    | Cliente -> Cérebro            | Envio da dica para o servidor.                      |
| ALL_TIPS LIST:<n1-d1,n2-d2...>                 | Cérebro -> Todos              | Cérebro exibe a lista compilada de dicas.           |

---

### Fase 3: Chat Causal e Sincronização

| Mensagem                           | Direção                     | Significado                                            |
|------------------------------------|-----------------------------|--------------------------------------------------------|
| CHAT_START TIME:<segundos>         | Cérebro -> Todos            | Inicia o cronômetro para discussão.                    |
| CHAT_MSG VT:<v1,v2,v3> MSG:<texto> | Cliente -> Cérebro -> Todos | Mensagem de texto trafegando com o Vetor de Timestamp. |
| VOTE_SKIP                          | Cliente -> Cérebro          | Jogador digita "/votar" para pular o chat.             |

| Mensagem                          | Direção          | Significado                                         |
|-----------------------------------|------------------|-----------------------------------------------------|
| SKIP_UPDATE COUNT:<atual>/<total> | Cérebro -> Todos | Atualização da barreira de sincronização (ex: 2/3). |
| CHAT_END                          | Cérebro -> Todos | Trava o terminal para iniciar a votação secreta.    |

---

## Fase 4: Veredito e Tratamento de Erros

| Mensagem                                                      | Direção            | Significado                                                                      |
|---------------------------------------------------------------|--------------------|----------------------------------------------------------------------------------|
| FINAL_VOTE TARGET:<nome_suspeito>                             | Cliente -> Cérebro | Voto final após o chat.                                                          |
| ROUND_END RESULT:<INOCENTES/INFILTRADO> SCORES:<p1-1,p2-2...> | Cérebro -> Todos   | Revela o vencedor da rodada e atualiza o placar.                                 |
| SYS_ERR MSG:<mensagem>                                        | Cérebro -> Todos   | Notifica sobre falhas parciais (ex: "O infiltrado desconectou! Reiniciando..."). |